Actividad 2: Desarrollo de código e introducción a AES

Teoría de Códigos y Criptografía

Adrián Molina Martínez
Alejandro Ortega Ramón
Gabijus Bagocius
Manuel Freniche Morales

Departamento de Informática

Universidad de Almería

Almería, 14 de octubre de 2026

Índice de Contenidos

0. Enunciado 1

Ejercicio 1: Aplicación de cifrado y descifrado con AES 1

Ejercicio 2: Mini-AES en modo CBC 6

Ejercicio 3: Documentación de la aplicación 13

0. Enunciado

Actividad 2. Desarrollo de código e introducción a AES

- Realizar una aplicación donde se muestre cómo se usa AES para cifrar y descifrar un fichero, y mensaje de texto. Generar la clave de sesión de forma aleatoria.
- Ilustrar los procesos de encriptado y desencriptado usando miniAES en modo CBC con la cadena 10101010111111010010111110000011, clave secreta K = 0101101011111101 y vector de inicialización IV = 1010101011111101.
- Documentar adecuadamente todos los procesos utilizados en el desarrollo de la aplicación, indicando toda la información necesaria que permita entender completamente el desarrollo del proyecto.

Ejercicio 1: Aplicación de cifrado y descifrado con AES

La aplicación forma parte del repositorio CRIPTOGRAFIA-26-27 del proyecto de voto electrónico y está escrita en Python 3.12 con interfaz de escritorio PyQt6. Cifra y descifra con AES, que cifra bloques de 128 bits con claves de 128, 192 o 256 bits. La aplicación usa claves de 256 bits (AES-256).

AES es un criptosistema simétrico: la misma clave sirve para cifrar y para descifrar. Por eso la aplicación gira alrededor de una *clave de sesión* que se genera de forma aleatoria y que el usuario tiene que conservar para poder descifrar después.

Ventana principal

La ventana tiene dos zonas:

- **Franja «Clave de sesión»**: campo con la clave (64 caracteres hexadecimales, es decir, 256 bits), botón *Generar clave nueva* (clave aleatoria) y botón *Copiar clave*. La aplicación no guarda la clave en ningún sitio, así que hay que copiarla para descifrar más tarde.
- **Pestañas «Mensaje» y «Fichero»**: cada una con su operación de cifrar y descifrar, y un panel de resultado que muestra la salida, el *nonce* usado y la clave usada.

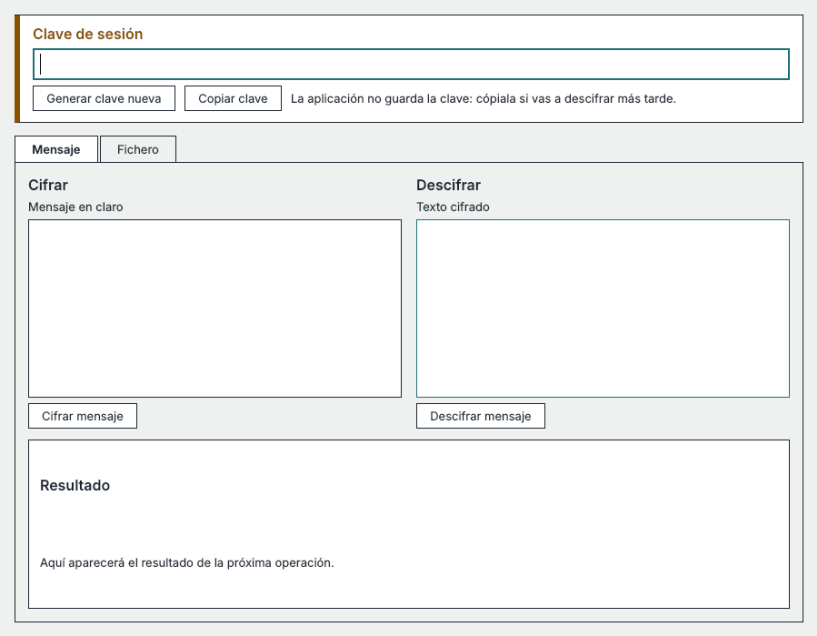


Figura 1: Ventana principal de la aplicación al arrancar

Generación aleatoria de la clave de sesión

Al pulsar *Generar clave nueva* se obtienen 32 bytes (256 bits) del generador de números aleatorios criptográficamente seguro del sistema operativo y se muestran en hexadecimal. En la prueba que se documenta se obtuvo la clave:

bc705c15fc8988b82227437b485f600098bae862da6e46ff7448fe5062c9ee6b

Cada pulsación produce una clave distinta e impredecible. Este es el único punto del programa del que salen claves y valores aleatorios (ver Ejercicio 3).

Cifrado y descifrado de un mensaje de texto

En la pestaña *Mensaje* se escribe el texto en claro y se pulsa *Cifrar mensaje*. Para la prueba se cifró el mensaje

Mi voto es: SÍ. Mesa electoral 3, Universidad de Almería.

que ocupa 59 bytes en UTF-8. El resultado es una línea en Base64 de 116 caracteres que corresponde a 87 bytes: 12 bytes de *nonce*, 59 de texto cifrado y 16 de etiqueta de autenticación.

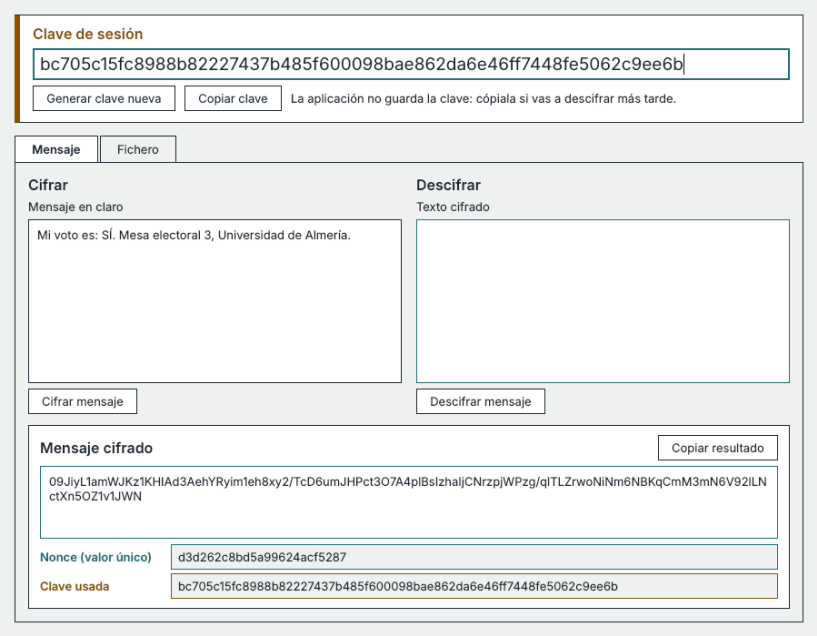


Figura 2: Cifrado de un mensaje de texto con la clave de sesión generada

Para descifrar se pega el texto cifrado en el recuadro *Texto cifrado*, con la misma clave en la franja superior, y se pulsa *Descifrar mensaje*. Se recupera exactamente el mensaje original, y el panel muestra el mismo *nonce* (d3d262c8bd5a99624acf5287) que se generó al cifrar, porque viaja dentro del propio texto cifrado.

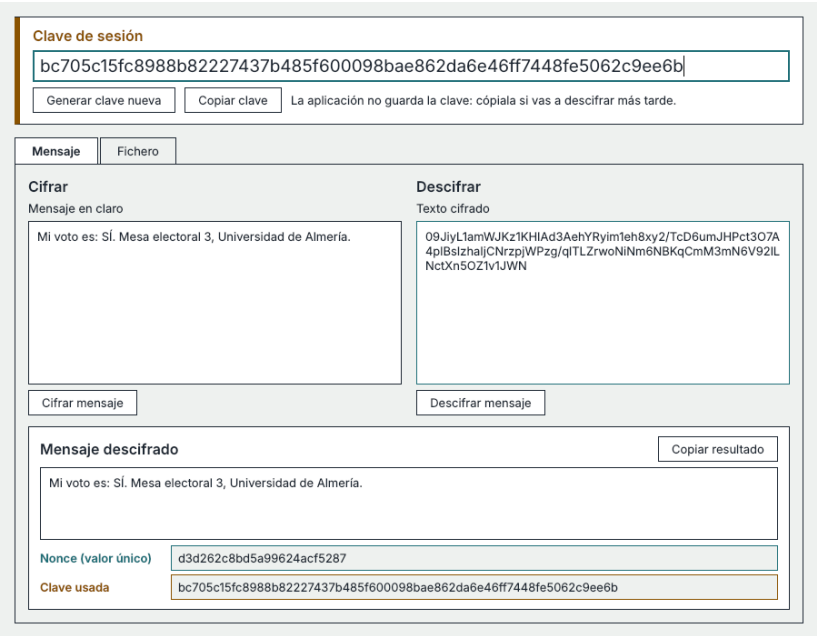


Figura 3: Descifrado del mensaje con la misma clave

Si se cifra otra vez el mismo mensaje con la misma clave, el resultado es distinto (en la prueba empezó por tWt42PQRbCMj. . . en lugar de 09JiyL1amWJK. . .), porque en cada

cifrado se genera un *nonce* nuevo. Así no se repiten patrones entre mensajes iguales.

Si se intenta descifrar con una clave distinta, la aplicación no muestra ningún contenido y avisa de que la clave es incorrecta o el dato está alterado:

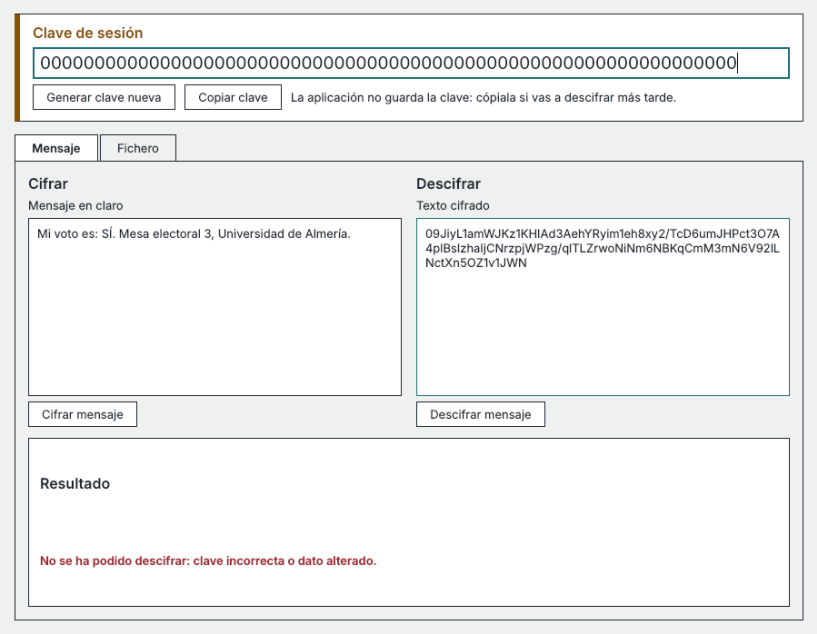


Figura 4: Intento de descifrado con una clave incorrecta

Cifrado y descifrado de un fichero

En la pestaña *Fichero* se elige el fichero de entrada y el destino, y se pulsa *Cifrar fichero*. Para la prueba se usó un fichero de texto papeleta.txt de 80 bytes, que se cifró en papeleta.txt.enc.



Figura 5: Cifrado de un fichero

El fichero cifrado mide 108 bytes, es decir, el original más 28 bytes. Sus primeros 12 bytes

son el *nonce* que muestra la aplicación:

```
$ od -A x -t x1 -N 16 papeleta.txt.enc
000000 9a 96 8c 13 3f 75 71 c4 2b a2 1d e0 90 99 f6 2d
```

Para descifrar se elige como entrada el fichero .enc, un destino nuevo y se pulsa *Descifrar fichero*:



Figura 6: Descifrado del fichero

El fichero descifrado es idéntico byte a byte al original:

```
$ cmp papeleta.txt papeleta_descifrada.txt && echo IDENTICOS
IDENTICOS
```

La aplicación nunca sobrescribe un fichero existente; si el destino ya existe, lo rechaza:

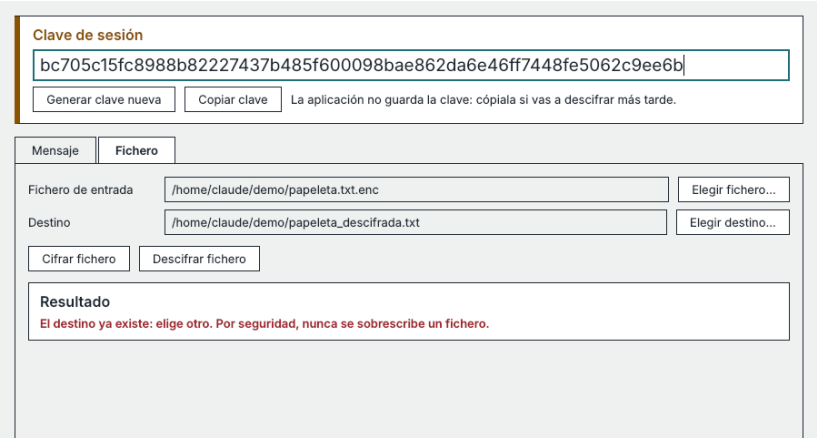


Figura 7: Aviso al elegir un destino que ya existe

Ejercicio 2: Mini-AES en modo CBC

Mini-AES

Mini-AES cifra bloques de 16 bits con una clave de 16 bits. Cada bloque, que escribimos $P = (p_0, p_1, p_2, p_3)$, se divide en cuatro bloques de 4 bits que se colocan en una matriz 2×2:

$$P = \begin{pmatrix} p_0 & p_2 \\ p_1 & p_3 \end{pmatrix}$$

Cada ronda aplica cuatro operaciones:

- **NibbleSub**: sustituye cada bloque de 4 bits según la caja S (Tabla 1).
- **ShiftRow**: rota la segunda fila de la matriz; la primera no se mueve. Es decir, $(b_0, b_1, b_2, b_3) \rightarrow (b_0, b_3, b_2, b_1)$.
- **MixColumn**: multiplica cada columna por una matriz fija, donde $\otimes$ es el producto de polinomios módulo $x^4 + x + 1$:

$$\begin{pmatrix} d_0 \\ d_1 \end{pmatrix} = \begin{pmatrix} 0011 & 0010 \\ 0010 & 0011 \end{pmatrix} \cdot \begin{pmatrix} c_0 \\ c_1 \end{pmatrix}$$

- **KeyAddition**: XOR con la clave de ronda k_i.

Se hacen dos rondas; la segunda no lleva MixColumn. Antes de la primera se suma la clave k_0. Para desencriptar se aplican las inversas en orden inverso y con las claves en orden inverso. KeyAddition, ShiftRow y MixColumn son su propia inversa, así que solo NibbleSub cambia por su inversa.

NibbleSub				NibbleSub^{-1}			
Input	Output	Input	Output	Input	Output	Input	Output
0000	1110	1000	0011	0000	1110	1000	0111
0001	0100	1001	1010	0001	0011	1001	1101
0010	1101	1010	0110	0010	0100	1010	1001
0011	0001	1011	1100	0011	1000	1011	0110
0100	0010	1100	0101	0100	0001	1100	1011
0101	1111	1101	1001	0101	1100	1101	0010
0110	1011	1110	0000	0110	1010	1110	0000
0111	1000	1111	0111	0111	1111	1111	0101

Tabla 1: Caja S de Mini-AES y su inversa

Modo CBC

El mensaje de 32 bits se divide en dos bloques de 16 bits. En modo CBC cada bloque se suma (XOR) con el bloque cifrado anterior antes de encriptarlo, y el primero con el vector de inicialización:

$$c_j = E_K(m_j \oplus c_{j-1}), \qquad m_j = D_K(c_j) \oplus c_{j-1}, \qquad c_0 = IV.$$

Los datos son:

$$m_1 = 1010\ 1010\ 1111\ 1101, \quad m_2 = 0010\ 1111\ 1000\ 0011,$$
$$K = 0101\ 1010\ 1111\ 1101, \quad IV = 1010\ 1010\ 1111\ 1101.$$

Generación de las claves de ronda

La clave K se divide en cuatro bloques de 4 bits, que forman la primera clave: $k_0 = (w_0, w_1, w_2, w_3) = (0101, 1010, 1111, 1101)$.

$k_1 = (w_4, w_5, w_6, w_7)$ es

$$\begin{aligned} w_4 &= w_0 \oplus \text{NibbleSub}(w_3) \oplus 0001 \\ &= 0101 \oplus \text{NibbleSub}(1101) \oplus 0001 \\ &= 0101 \oplus 1001 \oplus 0001 \\ &= 1101 \\ w_5 &= w_1 \oplus w_4 = 1010 \oplus 1101 = 0111 \\ w_6 &= w_2 \oplus w_5 = 1111 \oplus 0111 = 1000 \\ w_7 &= w_3 \oplus w_6 = 1101 \oplus 1000 = 0101 \end{aligned}$$

$k_2 = (w_8, w_9, w_{10}, w_{11})$ es

$$\begin{aligned} w_8 &= w_4 \oplus \text{NibbleSub}(w_7) \oplus 0010 \\ &= 1101 \oplus \text{NibbleSub}(0101) \oplus 0010 \\ &= 1101 \oplus 1111 \oplus 0010 \\ &= 0000 \\ w_9 &= w_5 \oplus w_8 = 0111 \oplus 0000 = 0111 \\ w_{10} &= w_6 \oplus w_9 = 1000 \oplus 0111 = 1111 \\ w_{11} &= w_7 \oplus w_{10} = 0101 \oplus 1111 = 1010 \end{aligned}$$

Por tanto:

$k_0 = 0101\ 1010\ 1111\ 1101, \quad k_1 = 1101\ 0111\ 1000\ 0101, \quad k_2 = 0000\ 0111\ 1111\ 1010$

Encriptado del bloque 1

Entrada (modo CBC):

$$\begin{aligned} X = m_1 \oplus IV &= 1010\ 1010\ 1111\ 1101 \oplus 1010\ 1010\ 1111\ 1101 \\ &= 0000\ 0000\ 0000\ 0000 \end{aligned}$$

Como m_1 coincide con el IV, la entrada es el bloque nulo.

Suma de la clave k_0:

$$\begin{aligned} A = X \oplus k_0 &= 0000\ 0000\ 0000\ 0000 \oplus 0101\ 1010\ 1111\ 1101 \\ &= 0101\ 1010\ 1111\ 1101 \end{aligned}$$

Ronda 1, NibbleSub:

$$\begin{aligned} B &= \text{NibbleSub}(0101), \text{NibbleSub}(1010), \text{NibbleSub}(1111), \text{NibbleSub}(1101) \\ &= 1111\ 0110\ 0111\ 1001 \end{aligned}$$

Ronda 1, ShiftRow:

$$\begin{aligned} C &= \text{ShiftRow}(1111\ 0110\ 0111\ 1001) \\ &= 1111\ 1001\ 0111\ 0110 \end{aligned}$$

Ronda 1, MixColumn:

$$C = \begin{pmatrix} 1111 & 0111 \\ 1001 & 0110 \end{pmatrix}$$

$$\begin{aligned} \begin{pmatrix} d_0 \\ d_1 \end{pmatrix} &= \begin{pmatrix} 0011 & 0010 \\ 0010 & 0011 \end{pmatrix} \cdot \begin{pmatrix} 1111 \\ 1001 \end{pmatrix} = \begin{pmatrix} (0011 \otimes 1111) \oplus (0010 \otimes 1001) \\ (0010 \otimes 1111) \oplus (0011 \otimes 1001) \end{pmatrix} \\ &= \begin{pmatrix} 0010 \oplus 0001 \\ 1101 \oplus 1000 \end{pmatrix} = \begin{pmatrix} 0011 \\ 0101 \end{pmatrix} \end{aligned}$$

$$\begin{aligned} \begin{pmatrix} d_2 \\ d_3 \end{pmatrix} &= \begin{pmatrix} 0011 & 0010 \\ 0010 & 0011 \end{pmatrix} \cdot \begin{pmatrix} 0111 \\ 0110 \end{pmatrix} = \begin{pmatrix} (0011 \otimes 0111) \oplus (0010 \otimes 0110) \\ (0010 \otimes 0111) \oplus (0011 \otimes 0110) \end{pmatrix} \\ &= \begin{pmatrix} 1001 \oplus 1100 \\ 1110 \oplus 1010 \end{pmatrix} = \begin{pmatrix} 0101 \\ 0100 \end{pmatrix} \end{aligned}$$

Por tanto, $D = 0011\ 0101\ 0101\ 0100$.

Ronda 1, KeyAddition:

$$\begin{aligned} E = D \oplus k_1 &= 0011\ 0101\ 0101\ 0100 \oplus 1101\ 0111\ 1000\ 0101 \\ &= 1110\ 0010\ 1101\ 0001 \end{aligned}$$

Ronda 2, NibbleSub:

$$\begin{aligned} F &= \text{NibbleSub}(1110), \text{NibbleSub}(0010), \text{NibbleSub}(1101), \text{NibbleSub}(0001) \\ &= 0000\ 1101\ 1001\ 0100 \end{aligned}$$

Ronda 2, ShiftRow:

$$\begin{aligned} G &= \text{ShiftRow}(0000\ 1101\ 1001\ 0100) \\ &= 0000\ 0100\ 1001\ 1101 \end{aligned}$$

Ronda 2, KeyAddition:

$$\begin{aligned} H = G \oplus k_2 &= 0000\ 0100\ 1001\ 1101 \oplus 0000\ 0111\ 1111\ 1010 \\ &= 0000\ 0011\ 0110\ 0111 \end{aligned}$$

Así pues, el bloque cifrado es $c_1 = H = 0000\ 0011\ 0110\ 0111$.

Encriptado del bloque 2

Entrada (modo CBC):

$$\begin{aligned} X = m_2 \oplus c_1 &= 0010\ 1111\ 1000\ 0011 \oplus 0000\ 0011\ 0110\ 0111 \\ &= 0010\ 1100\ 1110\ 0100 \end{aligned}$$

Suma de la clave k_0:

$$\begin{aligned} A = X \oplus k_0 &= 0010\ 1100\ 1110\ 0100 \oplus 0101\ 1010\ 1111\ 1101 \\ &= 0111\ 0110\ 0001\ 1001 \end{aligned}$$

Ronda 1, NibbleSub:

$$\begin{aligned} B &= \text{NibbleSub}(0111), \text{NibbleSub}(0110), \text{NibbleSub}(0001), \text{NibbleSub}(1001) \\ &= 1000\ 1011\ 0100\ 1010 \end{aligned}$$

Ronda 1, ShiftRow:

$$\begin{aligned} C &= \text{ShiftRow}(1000\ 1011\ 0100\ 1010) \\ &= 1000\ 1010\ 0100\ 1011 \end{aligned}$$

Ronda 1, MixColumn:

$$C = \begin{pmatrix} 1000 & 0100 \\ 1010 & 1011 \end{pmatrix}$$

$$\begin{pmatrix} d_0 \\ d_1 \end{pmatrix} = \begin{pmatrix} 0011 & 0010 \\ 0010 & 0011 \end{pmatrix} \cdot \begin{pmatrix} 1000 \\ 1010 \end{pmatrix} = \begin{pmatrix} (0011 \otimes 1000) \oplus (0010 \otimes 1010) \\ (0010 \otimes 1000) \oplus (0011 \otimes 1010) \end{pmatrix}$$
$$= \begin{pmatrix} 1011 \oplus 0111 \\ 0011 \oplus 1101 \end{pmatrix} = \begin{pmatrix} 1100 \\ 1110 \end{pmatrix}$$

$$\begin{pmatrix} d_2 \\ d_3 \end{pmatrix} = \begin{pmatrix} 0011 & 0010 \\ 0010 & 0011 \end{pmatrix} \cdot \begin{pmatrix} 0100 \\ 1011 \end{pmatrix} = \begin{pmatrix} (0011 \otimes 0100) \oplus (0010 \otimes 1011) \\ (0010 \otimes 0100) \oplus (0011 \otimes 1011) \end{pmatrix}$$
$$= \begin{pmatrix} 1100 \oplus 0101 \\ 1000 \oplus 1110 \end{pmatrix} = \begin{pmatrix} 1001 \\ 0110 \end{pmatrix}$$

Por tanto, $D = 1100\ 1110\ 1001\ 0110$.

Ronda 1, KeyAddition:

$$E = D \oplus k_1 = 1100\ 1110\ 1001\ 0110 \oplus 1101\ 0111\ 1000\ 0101$$
$$= 0001\ 1001\ 0001\ 0011$$

Ronda 2, NibbleSub:

$$F = \text{NibbleSub}(0001), \text{NibbleSub}(1001), \text{NibbleSub}(0001), \text{NibbleSub}(0011)$$
$$= 0100\ 1010\ 0100\ 0001$$

Ronda 2, ShiftRow:

$$G = \text{ShiftRow}(0100\ 1010\ 0100\ 0001)$$
$$= 0100\ 0001\ 0100\ 1010$$

Ronda 2, KeyAddition:

$$H = G \oplus k_2 = 0100\ 0001\ 0100\ 1010 \oplus 0000\ 0111\ 1111\ 1010$$
$$= 0100\ 0110\ 1011\ 0000$$

Así pues, el bloque cifrado es $c_2 = H = 0100\ 0110\ 1011\ 0000$.

Por tanto, el texto encriptado es

$$C = c_1\, c_2 = 0000\ 0011\ 0110\ 0111\ 0100\ 0110\ 1011\ 0000.$$

Desencriptado del bloque 1

Ronda 1, KeyAddition usando k_2:

$$\begin{aligned} G = c_1 \oplus k_2 &= 0000\ 0011\ 0110\ 0111 \oplus 0000\ 0111\ 1111\ 1010 \\ &= 0000\ 0100\ 1001\ 1101 \end{aligned}$$

Ronda 1, ShiftRow:

$$\begin{aligned} F &= \text{ShiftRow}(0000\ 0100\ 1001\ 1101) \\ &= 0000\ 1101\ 1001\ 0100 \end{aligned}$$

Ronda 1, NibbleSub^{-1}:

$$\begin{aligned} E &= \text{NibbleSub}^{-1}(0000), \text{NibbleSub}^{-1}(1101), \text{NibbleSub}^{-1}(1001), \text{NibbleSub}^{-1}(0100) \\ &= 1110\ 0010\ 1101\ 0001 \end{aligned}$$

Ronda 2, KeyAddition usando k_1:

$$\begin{aligned} D = E \oplus k_1 &= 1110\ 0010\ 1101\ 0001 \oplus 1101\ 0111\ 1000\ 0101 \\ &= 0011\ 0101\ 0101\ 0100 \end{aligned}$$

Ronda 2, MixColumn:

$$D = \begin{pmatrix} 0011 & 0101 \\ 0101 & 0100 \end{pmatrix}$$

$$\begin{aligned} \begin{pmatrix} d_0 \\ d_1 \end{pmatrix} &= \begin{pmatrix} 0011 & 0010 \\ 0010 & 0011 \end{pmatrix} \cdot \begin{pmatrix} 0011 \\ 0101 \end{pmatrix} = \begin{pmatrix} (0011 \otimes 0011) \oplus (0010 \otimes 0101) \\ (0010 \otimes 0011) \oplus (0011 \otimes 0101) \end{pmatrix} \\ &= \begin{pmatrix} 0101 \oplus 1010 \\ 0110 \oplus 1111 \end{pmatrix} = \begin{pmatrix} 1111 \\ 1001 \end{pmatrix} \end{aligned}$$

$$\begin{aligned} \begin{pmatrix} d_2 \\ d_3 \end{pmatrix} &= \begin{pmatrix} 0011 & 0010 \\ 0010 & 0011 \end{pmatrix} \cdot \begin{pmatrix} 0101 \\ 0100 \end{pmatrix} = \begin{pmatrix} (0011 \otimes 0101) \oplus (0010 \otimes 0100) \\ (0010 \otimes 0101) \oplus (0011 \otimes 0100) \end{pmatrix} \\ &= \begin{pmatrix} 1111 \oplus 1000 \\ 1010 \oplus 1100 \end{pmatrix} = \begin{pmatrix} 0111 \\ 0110 \end{pmatrix} \end{aligned}$$

Por tanto, $C = 1111\ 1001\ 0111\ 0110$.

Ronda 2, ShiftRow:

$$\begin{aligned} B &= \text{ShiftRow}(1111\ 1001\ 0111\ 0110) \\ &= 1111\ 0110\ 0111\ 1001 \end{aligned}$$

Ronda 2, NibbleSub^{-1}:

$$\begin{aligned} A &= \text{NibbleSub}^{-1}(1111), \text{NibbleSub}^{-1}(0110), \text{NibbleSub}^{-1}(0111), \text{NibbleSub}^{-1}(1001) \\ &= 0101\ 1010\ 1111\ 1101 \end{aligned}$$

Suma de la clave k_0:

$$\begin{aligned} X = A \oplus k_0 &= 0101\ 1010\ 1111\ 1101 \oplus 0101\ 1010\ 1111\ 1101 \\ &= 0000\ 0000\ 0000\ 0000 \end{aligned}$$

Salida (modo CBC):

$$\begin{aligned} m_1 = X \oplus IV &= 0000\ 0000\ 0000\ 0000 \oplus 1010\ 1010\ 1111\ 1101 \\ &= 1010\ 1010\ 1111\ 1101 \end{aligned}$$

Desencriptado del bloque 2

Ronda 1, KeyAddition usando k_2:

$$\begin{aligned} G = c_2 \oplus k_2 &= 0100\ 0110\ 1011\ 0000 \oplus 0000\ 0111\ 1111\ 1010 \\ &= 0100\ 0001\ 0100\ 1010 \end{aligned}$$

Ronda 1, ShiftRow:

$$\begin{aligned} F &= \text{ShiftRow}(0100\ 0001\ 0100\ 1010) \\ &= 0100\ 1010\ 0100\ 0001 \end{aligned}$$

Ronda 1, NibbleSub^{-1}:

$$\begin{aligned} E &= \text{NibbleSub}^{-1}(0100), \text{NibbleSub}^{-1}(1010), \text{NibbleSub}^{-1}(0100), \text{NibbleSub}^{-1}(0001) \\ &= 0001\ 1001\ 0001\ 0011 \end{aligned}$$

Ronda 2, KeyAddition usando k_1:

$$\begin{aligned} D = E \oplus k_1 &= 0001\ 1001\ 0001\ 0011 \oplus 1101\ 0111\ 1000\ 0101 \\ &= 1100\ 1110\ 1001\ 0110 \end{aligned}$$

Ronda 2, MixColumn:

$$D = \begin{pmatrix} 1100 & 1001 \\ 1110 & 0110 \end{pmatrix}$$

$$\begin{aligned} \begin{pmatrix} d_0 \\ d_1 \end{pmatrix} &= \begin{pmatrix} 0011 & 0010 \\ 0010 & 0011 \end{pmatrix} \cdot \begin{pmatrix} 1100 \\ 1110 \end{pmatrix} = \begin{pmatrix} (0011 \otimes 1100) \oplus (0010 \otimes 1110) \\ (0010 \otimes 1100) \oplus (0011 \otimes 1110) \end{pmatrix} \\ &= \begin{pmatrix} 0111 \oplus 1111 \\ 1011 \oplus 0001 \end{pmatrix} = \begin{pmatrix} 1000 \\ 1010 \end{pmatrix} \end{aligned}$$

$$\begin{pmatrix} d_2 \\ d_3 \end{pmatrix} = \begin{pmatrix} 0011 & 0010 \\ 0010 & 0011 \end{pmatrix} \cdot \begin{pmatrix} 1001 \\ 0110 \end{pmatrix} = \begin{pmatrix} (0011 \otimes 1001) \oplus (0010 \otimes 0110) \\ (0010 \otimes 1001) \oplus (0011 \otimes 0110) \end{pmatrix}$$
$$= \begin{pmatrix} 1000 \oplus 1100 \\ 0001 \oplus 1010 \end{pmatrix} = \begin{pmatrix} 0100 \\ 1011 \end{pmatrix}$$

Por tanto, $C = 1000\ 1010\ 0100\ 1011$.

Ronda 2, ShiftRow:

$$\begin{aligned} B &= \text{ShiftRow}(1000\ 1010\ 0100\ 1011) \\ &= 1000\ 1011\ 0100\ 1010 \end{aligned}$$

Ronda 2, NibbleSub^{-1}:

$$\begin{aligned} A &= \text{NibbleSub}^{-1}(1000), \text{NibbleSub}^{-1}(1011), \text{NibbleSub}^{-1}(0100), \text{NibbleSub}^{-1}(1010) \\ &= 0111\ 0110\ 0001\ 1001 \end{aligned}$$

Suma de la clave k_0:

$$\begin{aligned} X = A \oplus k_0 &= 0111\ 0110\ 0001\ 1001 \oplus 0101\ 1010\ 1111\ 1101 \\ &= 0010\ 1100\ 1110\ 0100 \end{aligned}$$

Salida (modo CBC):

$$\begin{aligned} m_2 = X \oplus c_1 &= 0010\ 1100\ 1110\ 0100 \oplus 0000\ 0011\ 0110\ 0111 \\ &= 0010\ 1111\ 1000\ 0011 \end{aligned}$$

Obtenemos $m_1\, m_2 = 1010\ 1010\ 1111\ 1101\ 0010\ 1111\ 1000\ 0011$, que es el mensaje original.

Ejercicio 3: Documentación de la aplicación

Requisitos

- Python ≥ 3.12.
- Dependencias de ejecución: cryptography (≥ 43.0), que aporta AES, y PyQt6 (≥ 6.7), que aporta la interfaz.

Arquitectura

El código está en src/evoting/ y se organiza en capas que solo dependen hacia abajo (ui → services → core, crypto, storage). Así la interfaz no hace criptografía y la criptografía no

sabe nada de la interfaz. La capa storage está reservada para fases posteriores del proyecto de voto.

Módulo	**Responsabilidad**
__main__.py	Punto de entrada (python -m evoting).
ui/app.py	Crea la aplicación Qt, aplica el estilo y abre la ventana.
ui/main_window.py	Ventana: franja de la clave y pestañas *Mensaje* y *Fichero*. Solo recoge datos, llama a los servicios y muestra el resultado o el aviso.
services/cipher.py	Casos de uso: generar clave, cifrar y descifrar mensajes y ficheros. Valida la entrada, convierte formatos (hexadecimal, Base64, UTF-8) y traduce los fallos a avisos para el usuario.
crypto/randomness.py	Única fuente de bytes aleatorios del proyecto.
crypto/aes_gcm.py	AES-256-GCM sobre bytes: generar clave, cifrar, descifrar y leer el *nonce*.
core/errors.py	DomainError: fallo con un mensaje fijo que se muestra al usuario.

Tabla 2: Módulos de la aplicación

Reglas de la capa criptográfica: no se implementan primitivas propias (se usa la librería cryptography), la aleatoriedad sale solo de randomness.py (el módulo random de Python está prohibido) y no se usan primitivas rotas como DES/3DES o el modo ECB.

Proceso 1: generación de la clave de sesión

La clave es de 256 bits y se obtiene del CSPRNG del sistema operativo mediante secrets.token_bytes:

```
# crypto/randomness.py
def random_bytes(n: int) -> bytes:
    if isinstance(n, bool) or not isinstance(n, int) or n <= 0:
        raise ValueError("n debe ser un entero positivo")
    return secrets.token_bytes(n)

# crypto/aes_gcm.py
KEY_SIZE: int = 32          # AES-256
def generate_key() -> bytes:
    return randomness.random_bytes(KEY_SIZE)

# services/cipher.py
def new_session_key() -> str:
    return aes_gcm.generate_key().hex()     # 64 caracteres hexadecimales
```

Cuando el usuario escribe o pega una clave, _parse_key comprueba que sean exactamente 64 caracteres hexadecimales y la convierte en 32 bytes. Si no, muestra «La clave está mal formada».

Proceso 2: cifrado con AES-256-GCM

Para cifrar se usa AES con clave de 256 bits en modo GCM (*Galois/Counter Mode*). Los modos clásicos (ECB, CBC, CFB, OFB) solo dan confidencialidad; GCM cifra en modo contador y además calcula una etiqueta de autenticación de 16 bytes. Esa etiqueta permite detectar al descifrar si la clave es incorrecta o si el dato se ha modificado, sin entregar nada en ese caso. Esto importa en el proyecto de voto, donde un dato manipulado no debe aceptarse.

Cada cifrado usa un *nonce* (valor único) de 12 bytes recién generado. Con GCM es imprescindible no repetir nunca el par (clave, *nonce*); por eso la función lo genera ella misma en lugar de recibirlo. Por el mismo motivo, dos cifrados del mismo mensaje con la misma clave dan resultados distintos.

```
# crypto/aes_gcm.py
NONCE_SIZE: int = 12
TAG_SIZE: int = 16
OVERHEAD: int = NONCE_SIZE + TAG_SIZE      # 28 bytes

def encrypt(key: bytes, plaintext: bytes) -> bytes:
    _check_key(key)
    nonce = randomness.random_bytes(NONCE_SIZE)
    return nonce + AESGCM(key).encrypt(nonce, plaintext, None)
```

El dato cifrado tiene siempre este formato:

nonce (12 B)	texto cifrado (*n* B)	etiqueta (16 B)

Por tanto mide el original más 28 bytes (en las pruebas: 59 → 87 y 80 → 108 bytes).

Proceso 3: descifrado

Se separa el *nonce* (primeros 12 bytes) y se pasa el resto a AES-GCM, que comprueba la etiqueta antes de devolver el contenido. Si no verifica, se lanza AuthenticationError y no se entrega nada, ni siquiera una parte:

```
def decrypt(key: bytes, data: bytes) -> bytes:
    _check_key(key)
    _check_data(data)                # al menos 28 bytes
    nonce = data[:NONCE_SIZE]
    try:
        plaintext = AESGCM(key).decrypt(nonce, data[NONCE_SIZE:], None)
    except InvalidTag:
        raise AuthenticationError("La etiqueta de autenticación no verifica: "
                                  "clave incorrecta o dato cifrado alterado.") from None
    return plaintext
```

La etiqueta no distingue entre clave incorrecta y dato alterado, así que la aplicación da un único aviso: «No se ha podido descifrar: clave incorrecta o dato alterado» (Figura 4).

Proceso 4: cifrado y descifrado de mensajes de texto

1. El texto se codifica en UTF-8 (admite tildes y la ñ).
2. Se cifra con el Proceso 2.
3. El resultado binario se codifica en Base64 en una sola línea, para que se pueda copiar y pegar como texto.

Al descifrar se hace al revés: se decodifica el Base64 (de forma estricta: se rechazan caracteres no válidos), se descifra con el Proceso 3 y se decodifica el UTF-8.

```
def encrypt_message(key: str, message: str) -> tuple[str, str]:
    plaintext = message.encode("utf-8")
    data, nonce_hex = encrypt_bytes(key, plaintext)
    return base64.b64encode(data).decode("ascii"), nonce_hex

def decrypt_message(key: str, ciphertext: str) -> tuple[str, str]:
    data = _decode_ciphertext(ciphertext)          # Base64 estricto
    plaintext, nonce_hex = decrypt_bytes(key, data)
    return plaintext.decode("utf-8"), nonce_hex
```

(Se muestra una versión simplificada; el código real además convierte los errores de codificación en avisos para el usuario.)

Proceso 5: cifrado y descifrado de ficheros

1. Se valida la clave antes de tocar ningún fichero.
2. Se lee el fichero de entrada completo en binario, por lo que sirve para cualquier tipo de fichero. Antes de leerlo se consulta su tamaño y se rechaza si supera 500 MiB (500 MiB + 28 B al descifrar).
3. Se cifra o descifra con los Procesos 2 o 3.
4. Se escribe el resultado en un fichero nuevo abierto en modo exclusivo ("xb"): si el destino ya existe, la apertura falla y no se sobrescribe nada. Si la escritura falla a medias, se borra el fichero incompleto.

```
def encrypt_file(key: str, source: Path, destination: Path) -> str:
    _parse_key(key)
    plaintext = _read_input(source, MAX_FILE_SIZE)
    data, nonce_hex = encrypt_bytes(key, plaintext)
    _write_new(destination, data)
    return nonce_hex

def decrypt_file(key: str, source: Path, destination: Path) -> str:
    _parse_key(key)
    data = _read_input(source, MAX_FILE_SIZE + aes_gcm.OVERHEAD)
    plaintext, nonce_hex = decrypt_bytes(key, data)
    _write_new(destination, plaintext)
    return nonce_hex
```

Al descifrar, si la etiqueta no verifica no se crea el fichero de destino.

Proceso 6: gestión de errores

Todo fallo provocado por el usuario se convierte en un DomainError con un texto fijo, que la ventana muestra en rojo en el panel de resultado. Los avisos nunca incluyen la clave, el contenido ni la ruta, para no filtrar información. Antes de cada operación se borra el resultado anterior, de modo que un fallo no deja a la vista datos de una operación previa.

Situación	**Aviso mostrado**
Sin clave	Falta la clave: genera una nueva o pega la que se usó al cifrar.
Clave que no son 64 hexadecimales	La clave está mal formada: deben ser 64 caracteres hexadecimales (0-9, a-f).
Texto cifrado que no es Base64	El texto cifrado está mal formado: está vacío, incompleto o tiene caracteres que no son Base64.
Dato de menos de 28 bytes	El dato cifrado está mal formado: está vacío o es demasiado corto.
Clave incorrecta o dato alterado	No se ha podido descifrar: clave incorrecta o dato alterado.
Fichero de entrada inexistente	El fichero de entrada no existe.
Fichero mayor de 500 MiB	El fichero supera el tamaño máximo admitido (500 MiB).
Destino existente	El destino ya existe: elige otro. Por seguridad, nunca se sobrescribe un fichero.

Tabla 3: Principales avisos de la aplicación